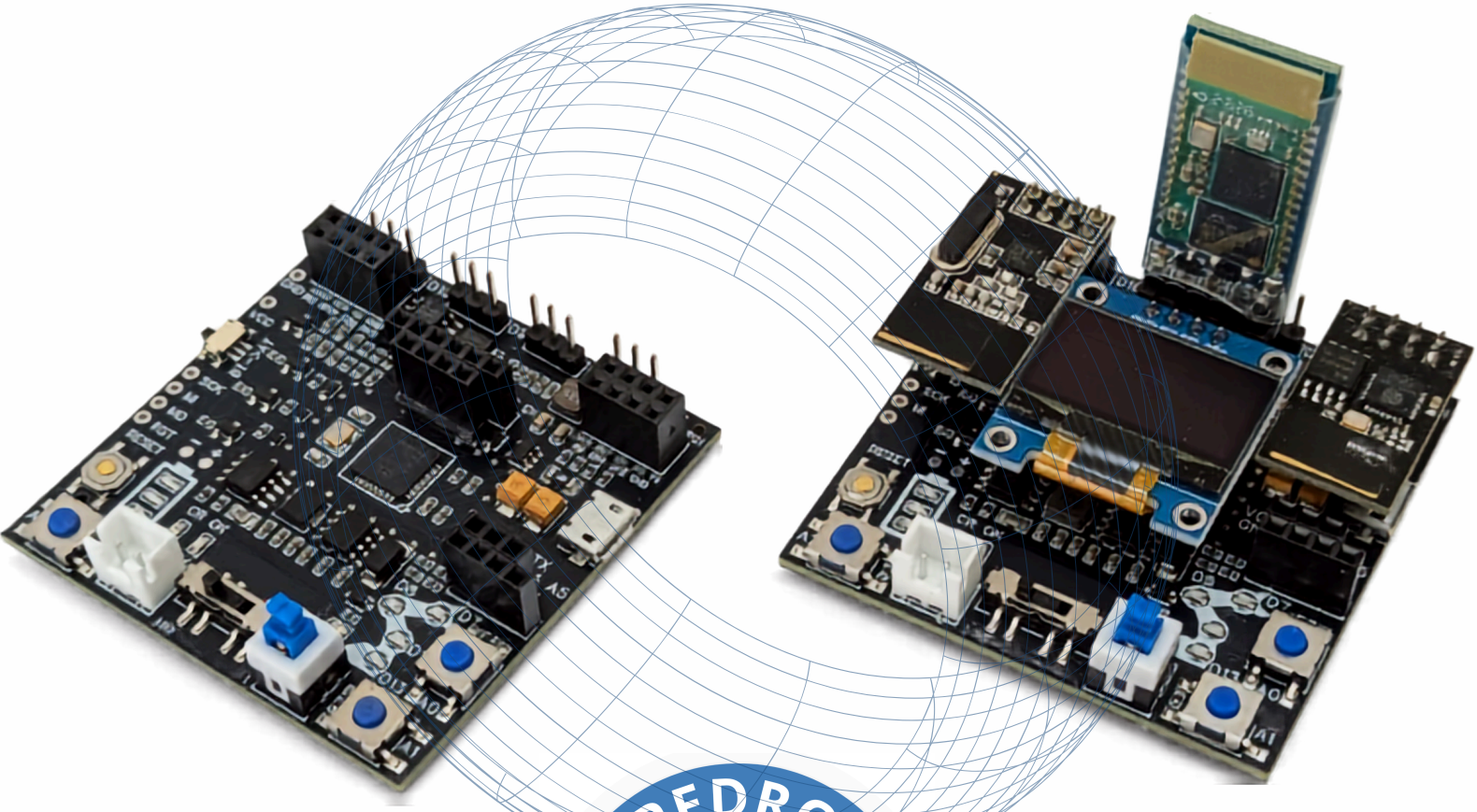
DÉBUTANT

pedro

90 min

STEM Program

Leçon #2 : La carte électronique





Complète la leçon et débloque ton nouveau badge

Colle ton badge sur ton passeport Pedro pour suivre ton évolution

Ta mission : découvre la carte électronique du robot Pedro.

1. Objectifs d'apprentissage

Dans cette nouvelle leçon, nous allons découvrir le fonctionnement de la carte électronique du robot Pedro. Considéré comme le cerveau de Pedro c'est elle qui reçois et traite toutes les instructions.

A la fin de cette leçon tu pourras coller ce nouveau badge sur ton passeport d'apprentissage.



2. Notes

- Aucune expérience en robotique n'est requise.
- Durée : ~90 minutes
- Matériels requis par élève ou groupe de deux :
  - une carte robot Pedro
  - un câble micro USB
  - un ordinateur avec l'IDE Arduino installé

3. Les systèmes embarqués

Un système embarqué est un système électronique programmable capable de réaliser des tâches spécifiques. Il combine à la fois une partie matérielle (hardware) et une partie logicielle (software).

Un robot est un ensemble de systèmes embarqués couplés a une structure mécanique mobile. Cela crée un système robotique capable de se déplacer ou de bouger dans son environnement de manière intelligente. En robotique on retrouve ces trois domaines combinés ensemble, à savoir :

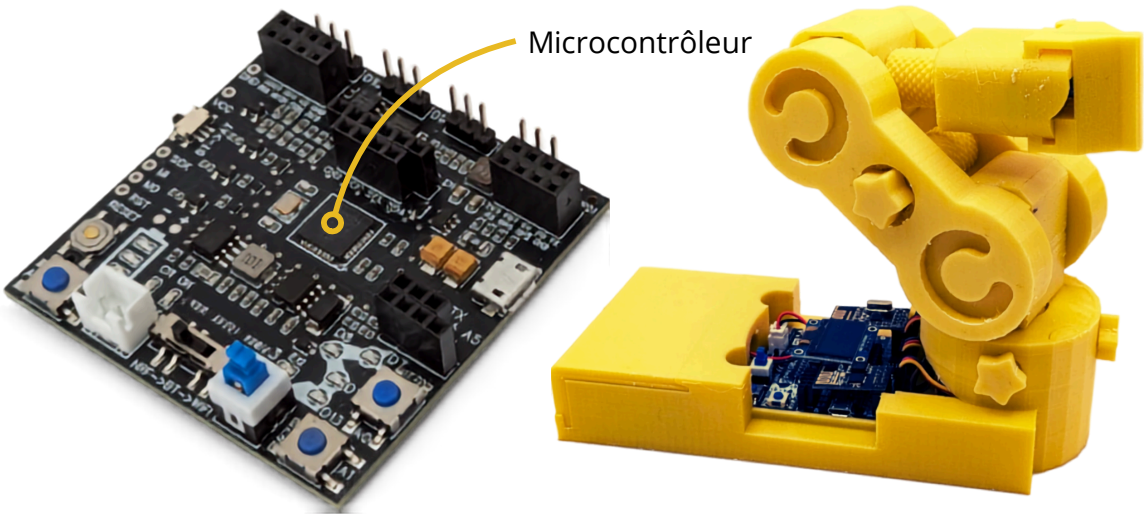
- ⚡ **L'électronique**
- 💻 **La programmation**
- ⚙️ **La mécanique**

De nos jours on retrouve des systèmes embarqués partout, nos équipements du quotidien sont pour la plus part équipés de systèmes intelligents.



4. Le cerveau du robot Pedro

Le robot Pedro étant un robot programmable, c'est un exemple de **système embarqué**. Il combine une carte électronique (hardware), un programme informatique (software) et un design entièrement imprimé en 3D (mécanique)



La carte Pedro est équipée d'un microcontrôleur **ATmega32U4**. Ce microcontrôleur joue le rôle de cerveau du robot. C'est lui qui traite les informations et ordonne le mouvement des servomoteurs.

C'est quoi un microcontrôleur?

Un microcontrôleur est un petit composant électronique capable de stocker des milliers d'instructions. Ces instructions sont codés depuis un ordinateur avant d'être transféré vers le microcontrôleur. Ce transfert ce fait en trois étapes:

1. l'écriture du code
2. la compilation du code
3. le transfert du code

La carte électronique joue le rôle de relai entre l'ordinateur et le microcontrôleur afin de transférer le programme. Pour pouvoir stocker toutes ces instructions le microcontrôleur est équipés de trois type de zone mémoires internes. Chaque zone mémoire possède une taille et est responsable du stockage de données spécifiques :

Type de Mémoire	Taille	Fonction
Mémoire Flash	**32 KO**	Enregistre le programme du robot
SRAM (RAM)	**2.5 KO**	Enregistre les données provisoires
Mémoire EEPROM	**1 KO**	Enregistre les données permanents

Un petit robot tel que Pedro possède une mémoire très limité comparé à un ordinateur où un smartphone :

- Un smartphone peut enregistrer des centaines de **Giga Octet** de données.
- Le microcontrôleur ATmega32U4 a seulement **36 Kilo Octet** d'espace de stockage.

A cause de cette limitation nous devons utiliser chaque zone mémoire avec précaution en se posant les trois questions suivantes:

- Quelles informations doit être stocké?
- Comment utiliser le moins d'octet possible?
- Comment optimisé au maximum l'écriture du code?

5. Questions (les systèmes embarqués)

1. Avec des mots simples décris ce qu'est un système embarqué?

_____

_____

2. Donne trois exemples du quotidien équipés de systèmes embarqués ?

_____

_____

3. Est ce qu'une voiture est équipé de systèmes embarqué?

_____

_____

4. Quel composant de la carte est considéré comme le cerveau de Pedro?

_____

_____

5. Qu’est qu’un microcontrôleur?

_____

_____

6. Quel est la taille totale d’un microcontrôleur ATmega32u4?

_____

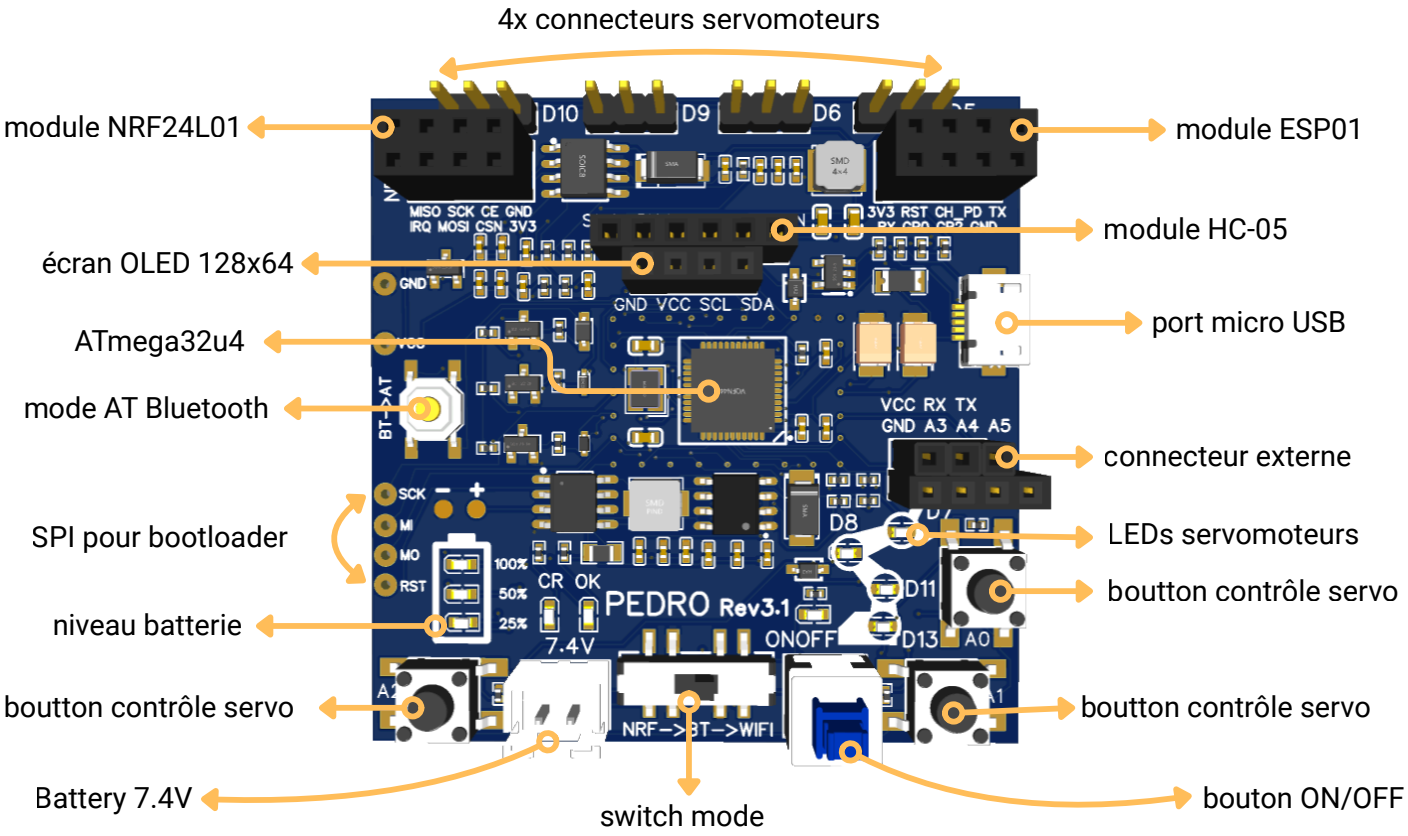
_____

6. Caractéristiques

La carte Pedro embarque tous les éléments nécessaires pour le contrôle du robot tel que, la gestion de la batterie, les connecteurs pour la communication à distance, les connecteurs des servomoteurs etc.

Elle est compatible Arduino donc programmable en langage C/C++ depuis l’IDE Arduino. La carte est équipé d’éléments suivants :

- un microcontrôleur ATmega32u4
- trois boutons programmables
- quatre connecteur pour servomoteurs
- un connecteur pour module NRF24L01 (communication radio)
- un connecteur pour module ESP01 (WiFi)
- un connecteur pour module HC-05 (communication Bluetooth)
- un connecteur pour écran OLED 128x64
- un indicateur du niveau de batterie
- un port micro USB
- une liaison SPI pour flasher le Bootloader
- un switch pour activer les mode radio, Bluetooth et WiFi
- un connecteur de batterie 7.4V CV
- un bouton ON/OFF



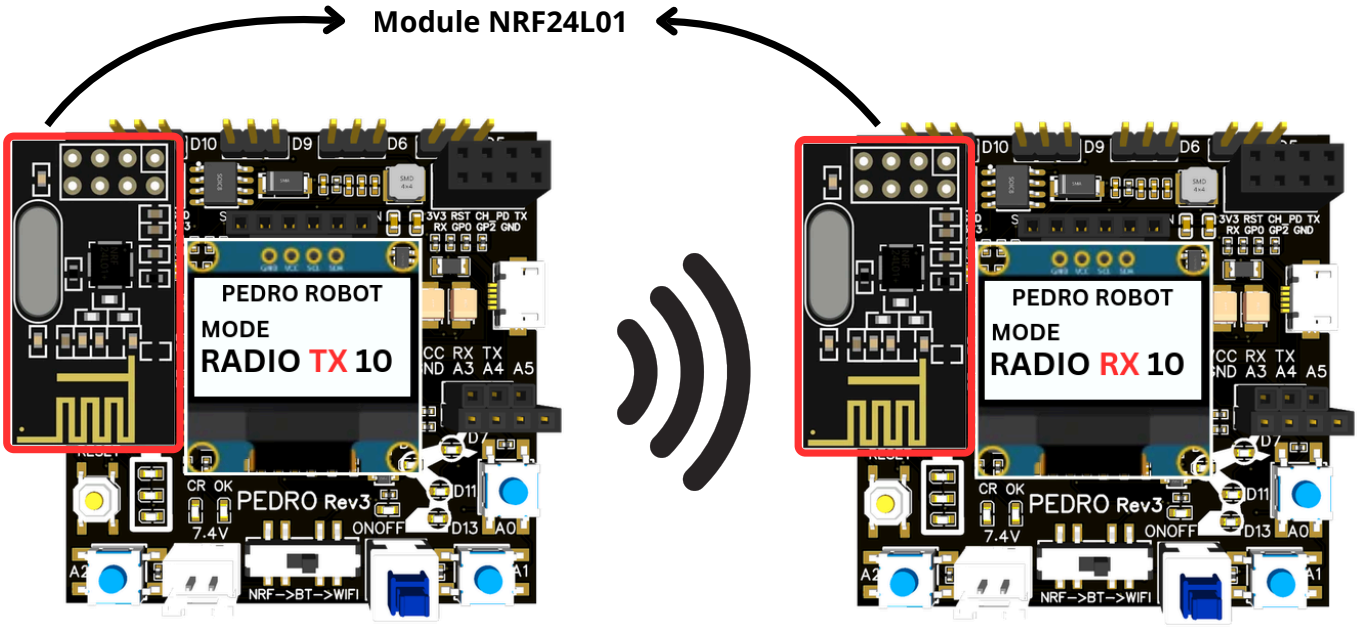
Le port micro USB à trois fonctionnalités:

1. **programmer la carte**
2. **recharger la batterie**
3. **contrôle USB**

Communication à distance

Deux modules permet au robot Pedro de communiquer à distance:

- le **module NRF24L01** : contrôle à distance entre deux ou plusieurs robots Pedro
- le **module HC-05** : contrôle à distance via smartphone



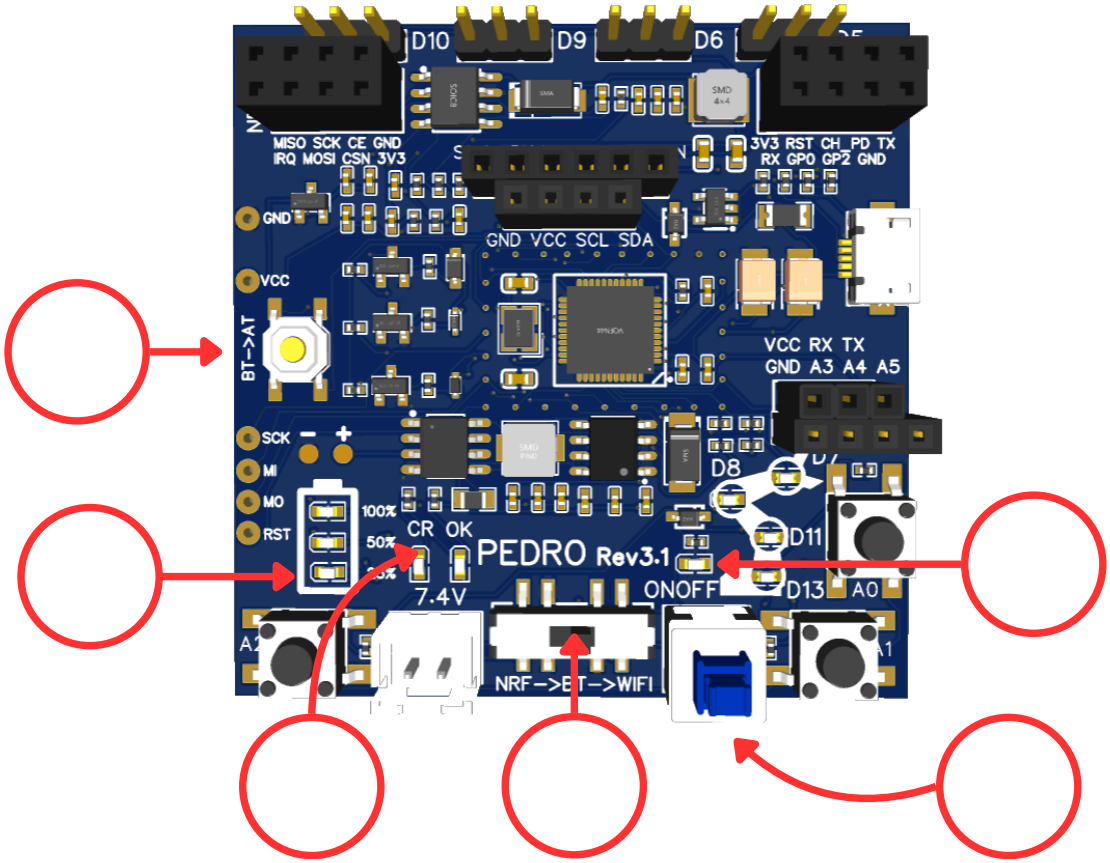
7. Identification des composants

Identifier les éléments de la carte en ajoutant les numéros correspondant.

Partie 1 : Les composants non programmable

Un composant non programmable est un composant qui n'est pas géré par le programme. Il peut fournir une indication ou interagir avec d'autre composant.

1- Bouton ON/OFF	2- LED ON/OFF	3- Niveau batterie
4- LEDs CR/OK	5- Buton AT mode Bluetooth	6- Switch mode

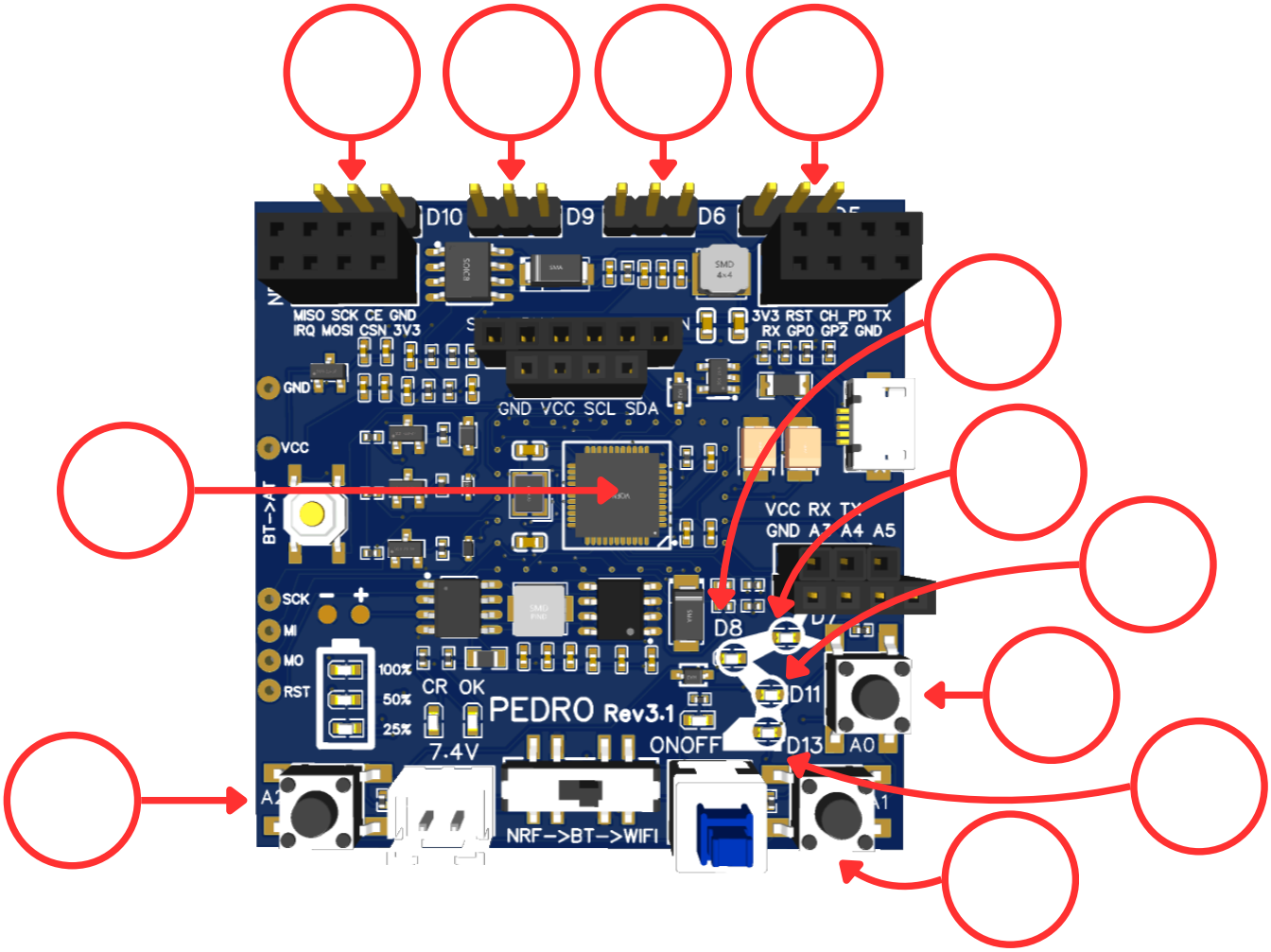


- *LED CR : Indique que la batterie est en cours de chargement.*
- *LED OK : Indique que la batterie est rechargée.*
- *Indicateur du niveau de la batterie : 25%, 50%, 100%.*

Part 2 : Les composants programmables

Un composant programmable est un composant qui peut être contrôlé par le programme.

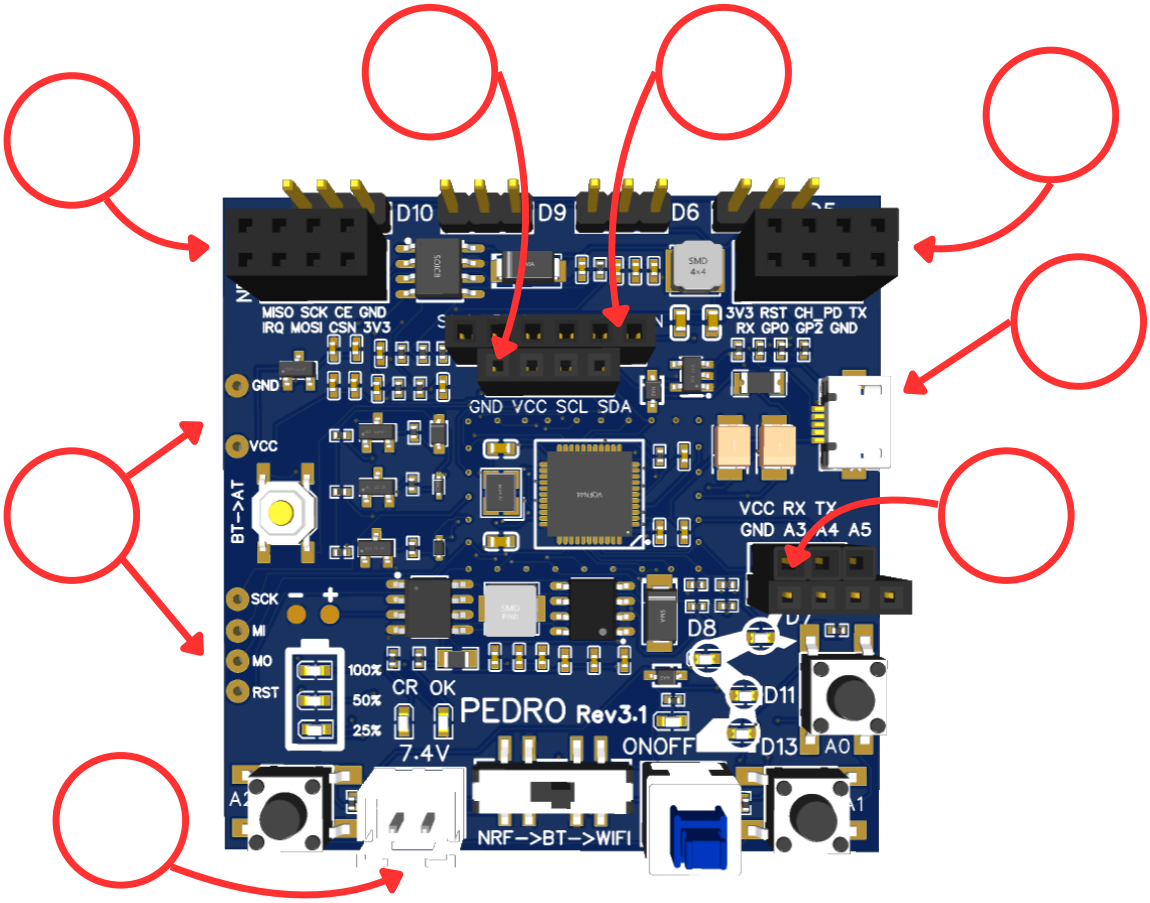
1- LED D13	2- LED D11	3- LED D8	4- LED D7
5- Servomoteur D5	6- Servomoteur D6	7- Servomoteur D9	8- Servomoteur D10
9- Bouton A0	10- Bouton A1	11- Bouton A2	12- ATmega32U4 Microcontrôleur



Partie 3 : Les connecteurs

Les connecteurs permettent de rajouter des équipements extérieurs afin d'interagir avec le robot.

1- Module radio NRF24L01	2- module WiFi ESP01	3- module bluetooth HC-05	4- écran OLED 128x64
5- Batterie 7.4V	6- Free Pin	7- port micro USB	8- SPI Bootloader



8. Arduino

L'IDE Arduino **(Integrated Development Environment)** est le logiciel qui permet d'écrire le code, le compiler et le transférer vers le microcontrôleur.

En utilisant l'IDE Arduino tu pourra écrire tes propres programmes afin de tester les éléments **programmable** vu plus haut tels que les LEDs, les boutons et les connecteurs.

Quatre étapes sont nécessaire pour programmer le robot Pedro :

1. **Ecrire le programme :** écrire le code dans l'IDE Arduino (C/C++).
2. **Compiler le programme :** l'IDE Arduino vérifie si le code n'a pas d'erreur.
3. **Transférer/Upload le programme :** Connecter Pedro avec le câble micro USB et cliquer sur **Téléverser** ou **Upload**. Le programme est transféré dans le microcontrôleur **ATmega32U4**.
4. **Exécution du programme :** à la fin du transfert le programme s'exécute automatiquement sur la carte.

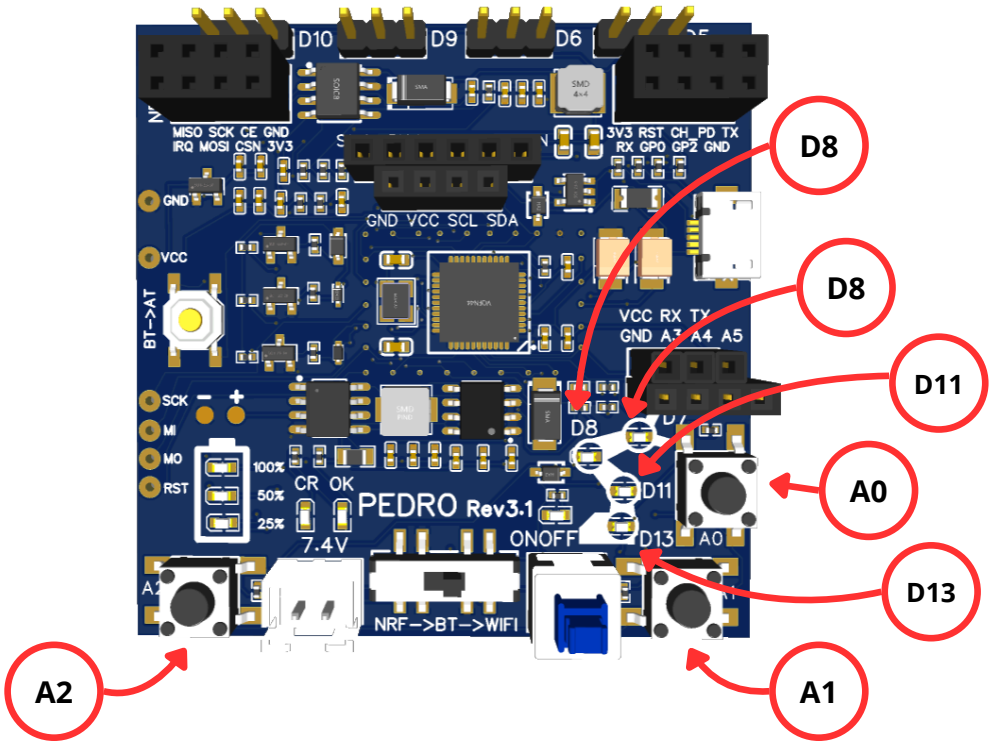
Chaque robot reçoit des informations (**inputs/entrées**) et exécute des actions (**outputs/sorties**).

- ☐ **Inputs** : ce sont les données reçu par le microcontrôleur. Le programme lit ces donnée en entrée avant d'exécuter une action. Exemples : appuyer sur un bouton, commande Bluetooth, commande radio etc.
- ☐ **Outputs** : ce sont les données envoyés par le microcontrôleur afin d'interagir avec son environnement. Exemples : allumer une LED, contrôler les servomoteurs etc.

Télécharger et installer la dernière version de l'IDE Arduino: https://www.arduino.cc/en/software

9. Programmation de la carte Pedro

Digital Outputs	Digital Inputs
• LED D13 • LED D11 • LED D8 • LED D7	• Button A0 • Button A1 • Button A2



Mots clé :

- **Programme** → ensemble d'instructions pour le robot.
- **Code** → Le texte écris dans le programme.
- **Transférer/Upload** → Transféré le programme de l'ordinateur vers le microcontrôleur.
- **Compiler** → Convertir le code en langage machine (compris par le microcontrôleur).
- **Microcontrôleur** → Le cerveau du robot qui exécute le programme.
- **Input/Entrée** → Information reçu par le robot.
- **Output/Sortie** → Action réalisé par le robot.

Programme 1 : les LEDs

Nous allons programmer la carte Pedro afin de contrôler les LEDs.

- Connecte la carte Pedro à l'ordinateur via le câble micro USB.
- Ouvre l'IDE Arduino
- Sur l'IDE sélectionne le port associé à la carte Pedro : **Outils → Port**
- Sélectionne le type de carte : **Outils → Carte → Arduino AVR Boards → Arduino Micro**
- Ouvre le programme Blink : **Fichier → Exemples → 01.Basics → Blink**
- Le programme Blink a deux fonctions **:**
  - la fonction **setup()**: s'exécute une seule fois au démarrage de la carte. Cette fonction configure la **LED D13** comme une sortie (output).
  - la fonction **loop()**: s'exécute en boucle, c'est le programme principale qui va nous permettre de contrôler les LEDs de la carte Pedro.

Chaque ligne de code joue un rôle dans le fonctionnement de la carte Pedro. On appelle cela des **lignes de code.**

```
// the setup function runs once when you press reset or power the board
void setup() {
  pinMode(13, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

- Sans modifier le programme clique sur **Compiler** puis **Téléverser** afin de transférer le programme Blink vers la carte Pedro.

☐ **Expérimentation 1** : contrôle de la LED D13

1. Après avoir téléversé le programme observe la carte Pedro, que remarques tu?

   _____

2. Quelle est le numéro de la LED qui s'allume?

   _____

3. Est-ce que la LED reste allumée ou clignote?

   _____

4. Modifie le programme Blink en remplaçant **OUTPUT** par **INPUT** puis compile et téléverse le programme.
5. Observe la carte Pedro, que remarques tu ?

   _____

6. En relisant le chapitre **8. Arduino**, explique le rôle des lignes de code suivantes sur le comportement de la LED :
   - **pinMode (13, OUTPUT)**

     _____

   - **pinMode (13, INPUT)**

     _____

7. Re-modifie le code afin de revenir au programme Blink initiale.
8. Modifie le programme Blink en remplaçant **HIGH** par **LOW** puis compile et téléverse le programme. Que remarques tu ?

   _____

9. Modifie cette fois le programme en remplaçant tous les **LOW** par **HIGH.** Que remarques tu ?

   _____

10. Par cette expérience sur le contrôle de la LED, explique le rôle des lignes de code suivantes sur le comportement de la LED :
    - **digitalWrite(13, HIGH)**

      _____

    - **digitalWrite(13, LOW)**

      _____

11. Re-modifie à nouveau le code afin de revenir au programme Blink initiale.
12. Compile et téléverse le programme. La LED re-clignote à nouveau.

☐ **Expérimentation 2** : contrôle de la durée

1. Modifie les deux lignes **delay(1000)** par **delay(500)** puis compile et téléverse le code. Est ce que la LED clignote plus vite ou plus lentement?

   _____

2. Modifie les lignes **delay(500)** par **delay(5000),** compile et téléverse le code. Que remarques tu ?

   _____

3. Observe le comportement de la LED en modifiant **delay** avec les valeurs suivantes : **100**, **2000**, **6000.**

☐ **Expérimentation 3** : contrôle des LED D7, D8, D11

1. Re-modifie le code afin de revenir au programme Blink initial.
2. Compile et téléverse le programme.
3. Modifie le numéro de la LED **13** par la LED **7** dans la fonction **setup** et **loop.**
4. Compile et téléverse le programme la LED **D7** se met à clignoter.
5. Modifie à nouveau le programme afin de tester les LEDs **D8** et **D11**.

☐ **Expérimentation 4** : contrôle de toutes les LEDs

Modifie le programme Blink afin que les quatre LEDs **(D7, D8, D11, and D13)** clignotent en même temps.

Programme 2 : contrôle de la LED par bouton

- Ouvre le programme Bouton : **Fichier → Exemples → 02.Digital → Button**

```
const int buttonPin = 2;  // the number of the pushbutton pin
const int ledPin = 13;    // the number of the LED pin
int buttonState = 0;  // variable for reading the pushbutton status

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

☐ **Expérimentation 1 :**

1. Modifie la ligne **buttonPin = 2** par **buttonPin = A0.**
2. Compile et téléverse le programme.
3. Appui sur le bouton **A0** de la carte Pedro. Que remarques tu ?

_____

4. Modifie la ligne **ledPin = 13** par **ledPin = 7.**
5. Compile et téléverse le programme.
6. Appui sur le bouton **A0** de la carte Pedro. Que remarques tu ?

_____

7. Modifie le code afin de contrôler la LED **D11** puis **D8** avec le bouton **A0.**
8. Modifie ensuite la ligne **buttonPin** par **buttonPin = A1.**
9. Compile et téléverse le programme.
10. Appui sur le bouton **A1** de la carte Pedro. Que remarques tu ?

_____

11. Modifie ensuite la ligne **buttonPin** par **buttonPin = A2.**
12. Compile et téléverse le programme.
13. Appui sur le bouton **A2** de la carte Pedro. Que remarques tu ?

_____

☐ **Expérimentation 2 :**

Modifie le programme bouton afin de contrôler toutes les LED comme ceci:

- Le bouton **A0** contrôle les LED **D7** et **D13** en même temps.
- Le bouton **A1** contrôle la LED **D8**.
- Le bouton **A2** contrôle la LED **D11**.

Quiz

1. Quel type de microcontrôleur est équipé le robot Pedro?

- [ ] ATmega32U4
- [ ] ESP32
- [ ] Raspberry Pi
- [ ] STM32

2. Quel logiciel est requis pour programmer Pedro??

- [ ] Visual Studio
- [ ] Arduino IDE
- [ ] Python
- [ ] Aucun

3. Quels modules permets à Pedro de communiquer à distance?

- [ ] module NRF24L01
- [ ] module HC-05
- [ ] module GPS
- [ ] Aucun

4. Quelle est la résolution de l'écran du robot Pedro?

- [ ] 64x64
- [ ] 128x32
- [ ] 128x64
- [ ] 256x128

5. Quel composant de la carte Pedro reçois et traite les informations?

- [ ] Les servomoteurs
- [ ] Le microcontrôleur
- [ ] La batterie
- [ ] Le module Bluetooth

6. Quel composant représente le cerveau de la carte Pedro?

- [ ] L'écran OLED
- [ ] Le microcontrôleur ATmega32u4
- [ ] Le chargeur de batterie
- [ ] Le module NRF24L01

7. Quelles sont les fonctions du port micro USB?

- [ ] Alimenter la carte seulement
- [ ] Programmer la carte
- [ ] Recharger la batterie
- [ ] Communiquer en USB

8. Quel module permets de contrôler Pedro avec un smartphone?

- [ ] NRF24L01
- [ ] HC-05
- [ ] ESP8266-01
- [ ] GPS module

9. Quel est l'espace de stockage du microcontrôleur ATmega32u4?

- [ ] 1 Go
- [ ] 36 Ko
- [ ] 10 Ko
- [ ] 100 Mo

10. Quel composant indique la recharge complète de la batterie?

- [ ] LED CR
- [ ] LED OK
- [ ] Battery Level LEDs
- [ ] OLED screen

🎯 Bravo, mission accomplie !

Tu viens de débloquer le badge **Pedro Electronics.**
Ajoute-le à ton passeport et rendez-vous à la prochaine leçon.

